Subscription marking — design

Date: 2026-10-13 · Status: approved in chat; revised after a 3-lens red-team review and verification rounds 2–5 (round 4 led to deleting silent no-note matching altogether; looping stopped at round 5 per the validator-loop rule); awaiting Alex's review of this written spec · Phase: 17

Goal

Alex knows when something is a subscription before Trim's detector does (the detector needs three charges). He wants to mark an expense as a subscription himself, including one he has already logged, and see it on /subscriptions straight away.

Today the only manual path is Quick Add's "Repeat this" checkbox. It is hidden in a collapsed section below the category chips, it only works at the moment an expense is first logged, and it auto-logs future charges every night. There is no way to mark an already-logged transaction, and the Subscriptions page has no add button even though its subtitle says "plus any you marked yourself". (Verified in code, 2026-10-13.)

Decisions (Alex, 2026-10-13)

Question	Decision
What happens after marking	Track only. Shows on Subscriptions immediately. Alex keeps logging the charges himself. Trim never creates a transaction for a tracked subscription.
Attaching the next charge	Ask each time. A matching expense triggers "Is this your Netflix subscription?". Yes attaches it. Nothing attaches silently.
Where to mark	Transactions → Edit dialog, and an "Add a subscription" button on /subscriptions.
Cadences	Monthly, weekly, yearly (new).

"Repeat this" stays as it is, for anything Alex wants auto-logged.

The rule that prevents double counting

The red team's main finding was that every loose end produced a second row for the same subscription, and both rows counted in "Per month". Four review rounds later, the rule that survived is short:

> Trim acts silently only on an exact name match. Anything it is less sure about, it asks.

An earlier draft also tried to silently match subscriptions without notes (same category, similar amount, aligned billing days). Rounds 2, 3 and 4 each found a new way for that to hide a different subscription. Two £10.99 no-note services billed three days apart cannot be told apart from data, and hiding one silently shrinks "Per month" with no way back. So that branch was deleted rather than patched again. The consequences:

1. Marking something Trim already knows by name doesn't create a second subscription.
   - Same note (the exact two-word key) as a tracked subscription → Trim offers "Add to Netflix?" / "Is Netflix back?". To keep them separate, Alex gives this expense a different note: two concurrent subscriptions with identical notes are treated as one (Known limits). The Edit dialog's note field is right there, so this is never a dead end.
   - Same note as a detected row → "Trim already spots this as Spotify" with a link to Subscriptions (or change the note).
   - A weaker resemblance (first word only, or no note) → Trim offers "Add to …?", and "It's a different subscription" goes straight through. The server never second-guesses a weak match.
   - `/track` enforces only the exact-name rule (409 with the existing key), so there is nothing to "decline" and no loop.
2. A detected row is hidden only when its exact note matches a tracked subscription's notes (active, or cancelled with no charge since the cancellation). It never hides a different subscription from the same brand ("Apple TV" stays next to a tracked "Apple Music"), and it never hides anything without a note.
3. Double submits and races cannot leave a second row. The link step reports which rows it actually linked. If the marked transaction is not among them, the new subscription is deleted and the request answers 409. Buttons disable while a request is in flight.

The accepted cost: a no-note tracked subscription whose charges Alex declined or missed three times can also appear as an "inferred" detected row. That is visible, and fixed with one tap ("Not a subscription" on the inferred row). The Quick Add prompt makes it rare.

What Alex will see

1. Transactions → Edit (any expense) → "Subscription" section. It is applied by the dialog's normal Save, after the field edits.
   - Not linked: a "This is a subscription" button. Tapping it asks the server about the expense as currently typed in the dialog (note, category, amount), not as last saved. Then either:
     - Same note as a tracked subscription: "Add to Netflix (Monthly · £10.99)?" — or, for a cancelled one, "Add to Netflix (cancelled)" for a charge from before the cancellation, or "Is Netflix back?" (reactivates it) for one after. To keep it separate, change the note; or
     - Similar but not the same note, or no note: the same offer, plus "It's a different subscription", which goes straight to the pills; or
     - Same note as a detected row: "Trim already spots this as Spotify" plus a link to Subscriptions (or change the note); or
     - Monthly / Weekly / Yearly pills, then "Other charges that look like this one". These come in two groups: likely ones, pre-ticked (same note, a similar amount, and spaced about one cycle apart), and "Maybe these too?", unticked (other look-alikes).
   - Linked to a subscription Alex marked: "Part of Netflix" and a "Remove from this subscription" toggle.
   - Linked to a "Repeat this" schedule: an information line, "Logged automatically by a repeating schedule — manage it on Subscriptions".
   - Income rows: no section.
2. Subscriptions → "Add a subscription" (header button). A dialog with a search box lists unlinked expenses from the last 13 months, newest first (long enough to reach a yearly charge). It has

its own server query. Tapping one leads to the same choices as above, then “Track it”. This dialog has no note field, so where the Edit dialog says “change the note”, it offers “Change its note in Transactions”, which opens `/transactions?edit=<id>` with that expense’s edit dialog open (Transactions gains that query parameter). A “Trim already spots this” result there says “It’s already on this page” and scrolls to the row.

3. After logging a matching expense in Quick Add, a toast asks “Is this your Netflix subscription?” with Yes and No, for about 12 seconds.
   - For an active subscription it only appears when the expense is dated near an expected charge date: within ±1 day weekly, ±5 days monthly, ±21 days yearly, of the last attached charge plus a whole number of cycles. A missed month still lines up with the next one.
   - It is the last toast fired, so it sits on top of the stack.
   - The Toaster gets `pointer-events: auto`, so the buttons stay tappable even while another dialog is open.
   - For a cancelled tracked subscription, an expense dated after the cancellation with the exact same note gets “Is Netflix back?” (any date; a resubscription starts on a new billing day). Yes marks it active and attaches the charge. Anything weaker never asks about a cancelled subscription, or every “Amazon …” order would ask about a cancelled Prime forever.
   - Unnamed subscriptions (marked from a no-note expense) use the existing label fallback, e.g. “Is this your Monthly £10.99 Entertainment subscription?” — never “null”.
4. Subscription rows. Rows Alex marked get a “Marked by you” tag. “Repeat this” rows, which today say “Manually marked”, are relabelled “Logs itself”. Marked rows offer Mark cancelled / Mark active, and Remove, with a confirm. Remove deletes the subscription only; its transactions stay, unlinked. Marked rows have no amount edit: their amount always follows the latest attached charge (see below). Their “per month” stat reads “based on your latest charge”.
5. Transaction rows linked to a marked subscription show a “Subscription” pill. “Repeat this” rows keep their “Recurring” pill. The existing Recurring filter chip now includes both kinds (any row linked to a subscription), so it shows every subscription charge.
6. Cadence labels say “Yearly” instead of “Annual” everywhere (`CADENCE_LABELS`), matching the new pill.

Copy follows house rules: friendly, never red. Marking toast: “Tracking it as a subscription [repeat icon]” / “See it on Subscriptions.”

Data model

Reuse the existing `recurrences` table rather than adding a table. A tracked subscription is a `recurrences` row that the nightly job must never fire.

New migration **`server/migrations/017a_subscription_tracking.sql`**:

```
-- auto_log: true  = "Repeat this" (the nightly job inserts each charge). Every existin
--           false = tracked only (Alex logs charges himself; the job must never fire i
alter table public.recurrences
  add column if not exists auto_log boolean not null default true;

-- Tracked rows have no "next run". Their next expected date is derived from the attach
-- charges at read time. A NULL next_run_at can never satisfy the job's `<=` filter: a
-- independent guard beside auto_log.
```

```
alter table public.recurrences alter column next_run_at drop not null;
alter table public.recurrences drop constraint if exists recurrences_next_run_required;
alter table public.recurrences
  add constraint recurrences_next_run_required check (not auto_log or next_run_at is no

-- Yearly.
alter table public.recurrences drop constraint if exists recurrences_interval_check;
alter table public.recurrences
  add constraint recurrences_interval_check check (interval in ('monthly', 'weekly', 'a
```

- **017a**, not **020**: `server/test/encryptionScope.test.js:179` requires `019_encryption_drop_plaintext.sql` to sort last. Verified against the live database, 2026-10-13: 014–017 are applied; 012, 018, 018a and 019 are not (no `*_enc` column and no barrier function exist). 017a is independent of those migrations, so the live order (017a now, encryption later) and the replay order are both valid.
- **`annual`** in the database, because detected rows and the merchant-key regex already use it. The UI says "Yearly".
- Replays cleanly: every statement is idempotent. The constraint name was read from the live database. A real-PostgreSQL test proves it (see Tests).
- No encryption work: one boolean and a cadence word. `recurrences.amount` and `description` are already registered in `lib/encryptedFields.js`.
- No **`anchor_day`**. An earlier draft stored a billing day. Tracked rows no longer need one (next date = last charge + cycle, below), and the "Repeat this" drift it would also have fixed is an existing, documented edge case outside this feature.

Derived, never stored (one number, one definition)

For a tracked row, every figure comes from its attached charges at read time, so editing, re-dating or deleting a charge can never leave a stale number:

- `lastCharged` = date of the latest attached charge, or null when none is attached.
- `amount` = amount of the latest attached charge. With none attached, it falls back to the stored `recurrences.amount`.
- `nextExpected` = `lastCharged + cadenceDays` (7 / 30 / 365). With none attached it is null and is never computed; `addDays(null, …)` would throw and 500 the whole page. This is the same rule detected rows already use (`lib/subscriptions.js` `finaliseDetection`), now exported and shared.
- `occurrences` / `totalPaid` = count and sum of attached charges (0 when none are attached).
- `monthlyCost` / `annualCost` come from one pure helper, `subscriptionCost(amount, cadence)`: monthly ×1/×12, weekly ×52/12 / ×52, annual ÷12 / ×1. "Repeat this" rows use it too. (Today any non-monthly manual row is costed as weekly, which would price a yearly one at 52×.)
- Plus a single predicate, `isTracked(r) = r.auto_log === false`. It fails closed: a row with auto_log missing is not treated as tracked. Every guard and label uses it.

`CADENCE_DAYS`, `subscriptionCost` and `isTracked` live in **`server/lib/recurrences.js`** (pure, no database import), so the route, the matcher and `devMock` all import the same code. (`routes/subscriptions.js` imports `lib/supabase.js`, which exits without `SUPABASE_URL`, so devMock must never import from a route.)

The matcher — one home

New pure module **`server/lib/subscriptionMatch.js`**, unit-tested. Every caller uses it.

`prototypeOf(subscription)` = `{ ownKey, keys, category_id, amount, cadence, lastCharged }`: - ownKey is the merchant key (`normaliseMerchant`, the single normaliser in `lib/merchant.js`) of the note the subscription was marked with; - `keys` adds the keys of every attached charge, so a note that changed over time ("Spotify" → "Spotify Premium") is still recognised when asking; - `amount` and `lastCharged` are the derived figures.

Only **`ownKey`** is ever used silently. Absorbed `keys` are asking-only (round 5: attaching a lazily noted "Apple" charge to "Apple Music" must never make "apple" an exact key that silently hides iCloud).

Dates. `cancelled_at` is a UTC instant. Every comparison with a transaction date uses the cancellation day in the user's own zone. The matcher stays pure: it takes that day as an argument and never imports **`lib/userZone.js`**, which imports `lib/supabase.js` and would kill devMock. Routes compute it with `dayInZone(await userTimeZone(uid), new Date(cancelled_at))`, and devMock passes its mock zone. Transaction dates are already the user's local days.

`nameStrength(expenseOrRow, proto)` → `'exact' | 'firstWord' | 'none'`: - `'exact'`: the item's two-word key is in `proto.keys` (own or absorbed); - `'firstWord'`: otherwise, its first word equals the first word of any key; - `'none'`: different names, or either side has no key.

A detected row's key is its `merchantKey`; a synthetic `auto:` key counts as having no key.

`isSameSubscription(item, proto)` — the only SILENT match: the item's key equals **`proto.ownKey`** and amounts are within ±30%. Nothing else is ever silent: not absorbed keys, not first-word, not no-note. Used only for hiding a detected row and for preview's "Trim already spots this".

`isExactNote(item, proto)` — `nameStrength === 'exact'` at any amount. Used for `/track`'s 409 and preview's `strength: 'exact'`. These are not silent: they turn into an offer Alex confirms. A price jump (Disney+ £1.99 → £7.99) must still be recognised as the same subscription, or nothing could ever attach the new charge and marking it would create a duplicate (round 5).

`looksLikeSameCharge(expense, proto)` — ASKING paths only. The expense must be an expense with no `recurrence_id`. Then either: - `nameStrength` is `'exact'` (any amount: a person confirms it), or `'firstWord'` with amounts within ±30%; or - the expense has no note (whatever the subscription has): same `category_id`, amounts within ±20% (wide enough to catch a typical price rise, because a person confirms it). A named expense never matches an unnamed subscription: Alex named it, so it is probably something else.

`cycleAligned(date, lastCharged, cadence)`: for some k in ±1…±N, |days − k·cadenceDays| ≤ the tolerance:

Cadence	Tolerance	N
weekly	±1	4
monthly	±5	12
annual	±21	3

False when `lastCharged` is null.

Ranking (both pickers): exact > firstWord > no-note; then active over cancelled; then the closest amount.

`bestTrackedMatch(tx, trackedSubs)` — the Quick Add toast, at most one: - active, with ≥1 attached charge: `looksLikeSameCharge` and `cycleAligned`. Weekly only on `exact` / `firstWord` (a no-note weekly match would fire most days); - cancelled: `nameStrength === 'exact'`, dated after the cancellation day, no cycle check → `kind: 'reactivate'`; - zero-charge subscriptions never toast.

`bestPreviewMatch(expense, trackedSubs)` — preview's "Add to …?" offer, at most one. The same ranking, but no cycle check, zero-charge included, because Alex is choosing deliberately: - active → `kind: 'attach'`; - cancelled, expense on or before the cancellation day → `kind: 'attach'` (an old charge of a cancelled service, with no reactivation); - cancelled, after → `kind: 'reactivate'`.

Plus `strength` (`'exact'` if `isExactNote`, else `'weak'`), which tells the client whether "It's a different subscription" is offered (weak only; exact needs a different note).

A charge after a cancellation, exact note (a final charge in a notice period, or a resubscription): preview offers both "Add to Gym (final charge)" (attach without reactivating) and "Is Gym back?" (reactivate). The toast does the same: its two buttons are "It's back" and "Final charge", and ignoring it does nothing.

Detector primitives only. To compare a detected row, `finaliseDetection` gains one primitive, `lastAmount`. It never returns a transaction object. `GET /api/subscriptions` builds detected rows field by field instead of spreading **`...`**`d`, so no decoded row can reach the browser (BUGS-FIXED 2026-09-23 #5).

API

Everything sits under the existing `/api/subscriptions` mount, so `server/index.js` and `routeMountTable.test.js` do not change. Every query is scoped by `req.user.id` (the server holds the service-role key, so RLS protects nothing), every id is ownership-checked, and every recurrence read that feeds a guard selects **`auto_log`**. Reads use `selectFor` + `decodeRow(s)`. Responses are built field by field and never return raw decoded rows. Every insert, update and delete whose result is read chains **`.select(...)`**, because real supabase-js returns `data: null` without it.

Six new endpoints:

1. `GET /api/subscriptions/track/preview`

Query: `transactionId` (uuid), optional `description`, `categoryId`, `amount` (the dialog's current values; omitted ones fall back to the saved row), and optional `interval`.

The transaction must be the user's, an expense and unlinked (404 / 400 / 409). A missing category (on the dialog's current values) is reported as `needsCategory: true`, so the section can say "Give it a category first" before Save, not after the PATCH. Returns: `{ existing: SubRef|null, detected: SubRef|null, likely: Cand[], maybe: Cand[] }`, where `SubRef`

= { merchantKey, name, amount, cadence, category } (enough for the client's subscriptionLabel) and Cand = { id, date, amount, description }.

- existing: bestPreviewMatch, returned as SubRef & { kind: 'attach'|'reactivate', strength: 'exact'|'weak' }.
- detected: a non-dismissed detected row (detection re-run on the paged unlinked expenses) that isSameSubscription the expense (exact note only).
- likely (only when interval is given): unlinked expenses on the exact-name path that are cycleAligned with the transaction. maybe: other looksLikeSameCharge hits. At most 24 in total, newest first.

2. **GET /api/subscriptions/track/recent?q=**

The user's unlinked expenses from the last 13 months (relative to the user's own day, via lib/userZone.js). The order matters: read the whole 13-month window, bounded by date, not by row count, paged by offset, advancing by the number of rows each page actually returned, ordered by **date** then **id**, until an empty** page. PostgREST silently caps a response at the project's max-rows, whose live value is not assumed: advancing by rows-returned and stopping only on an empty page is correct for any cap, and the **id** tie-breaker keeps page edges stable. A write landing between pages can still shift one row, so results are de-duplicated by id; a row skipped that way reappears on the next load, which is acceptable for a list; decode it; apply **q** (note or category name, in JS, since descriptions are encrypted once Phase 9.5 is live); then** take the newest 200. Limiting first and filtering second is the trap lib/merchant.js documents (0 matches instead of 3). Shape: Cand plus categoryId.

3. **POST /api/subscriptions/track**

Body { transactionId, interval: 'monthly'|'weekly'|'annual', includeTransactionIds?: uuid[] (≤100) }. 1. Load the transaction (scoped). 404 if missing; 400 if income; 400 "Give this expense a category first" if category_id is null; 409 if already linked. 2. If a tracked subscription isExactNote it (exact note, any amount; active, or cancelled at any date) → 409 { error, existing: SubRef & { kind, strength: 'exact' } }, with kind computed as in bestPreviewMatch. The client handles a 409 the same way in both dialogs: it shows that offer. Weak resemblances never 409, so "It's a different subscription" always goes through, and two different no-note services are always trackable. There is no "declined key" to carry, and therefore no loop. 3. Insert the recurrence via encodeWrite with .select('id').single(): auto_log: false, next_run_at: null, interval, category_id, type: 'expense', description and amount from the transaction. 4. Link with one update: .update({ recurrence_id }) .in('id', [transactionId, ...include]) .eq('user_id', uid).eq('type', 'expense').is('recurrence_id', null).select('id'). If it errors, or the marked transaction is not among the returned ids, delete the new recurrence (best effort) and answer 409. is_recurring is not set; that flag keeps meaning "created by a Repeat-this schedule". 5. 201 { subscription: { merchantKey: 'manual:<id>', linkedCount } }, with linkedCount taken from the returned rows. No XP.

4. **`POST /api/subscriptions/:merchantKey/attach`**

Body `{ transactionId, reactivate?: boolean }`. `manual:` keys only (400). The recurrence must be the user's (404) and `isTracked` (400 — "Repeat this" schedules are not hand-edited). The link is a conditional update, `.eq('user_id', uid).eq('type','expense').is('recurrence_id', null) .select('id')`, and zero rows means 409. `reactivate: true` also clears `cancelled_at` (the "Is it back?" path). `{ ok: true }`. No XP.

5. **`POST /api/subscriptions/:merchantKey/detach`**

Body `{ transactionId }`. Same key and ownership checks; the recurrence must be `isTracked`. The transaction must be linked to this recurrence: the update is `.eq('recurrence_id', recId).eq('user_id', uid).select('id')`, and zero rows means 404.

6. **`DELETE /api/subscriptions/:merchantKey`**

`manual:` keys whose recurrence `isTracked` only. "Repeat this" returns 400 ("cancel it instead", the unchanged rule). It first nulls `recurrence_id` on the linked transactions, scoped by user. The FK's `on delete set null` would do this in Postgres, but the test harness has no FKs, so the route does not rely on it. Then it deletes the row. 204.

Changed: **`GET /api/subscriptions`**

- The all-expenses read is paged by offset, advancing by the number of rows each page actually returned, ordered by **`date`** then **`id`**, until an empty** page** (as in `track/recent`). It is ordered ascending, so a silent 1,000-row cap would drop the newest charges, exactly the ones the derived `lastCharged` and `amount` now depend on. Preview's detection re-run uses the same paged read.
- Select `auto_log` on recurrences. Manual rows gain `tracking: isTracked(r)` and keep `source: 'manual'`.
- Tracked rows use the derived figures above. "Repeat this" rows keep `next_run_at` and their stored amount, costed through `subscriptionCost`.
- Suppression (exact note only): drop a detected row when it `isSameSubscription` the prototype of a tracked subscription that is:
  - active; or
  - cancelled, provided the detected row's `lastCharged` is on or before the cancellation day (converted to the user's zone). The old, unattached charges of a cancelled service must not reappear as an active detected row, or cancelling would not lower "Per month". A detected charge after the cancellation date means it came back, so it is shown.

  Synthetic (`auto:`, no-note) detected rows are never suppressed. Detected rows are built field by field (see the matcher).

Changed: **`PATCH /api/subscriptions/:merchantKey`**

Tracked rows accept `{ status: 'active'|'cancelled' }` only. `amount` → 400 ("follows your latest charge"). `dismissed` stays 400 (Remove is the off-ramp). "Repeat this" rows are unchanged.

Changed: **`POST /api/transactions`**

- The response gains `subscriptionMatch: (SubRef & { kind: 'attach'|'reactivate' }) | null`, from `bestTrackedMatch`. It is computed only for an expense without the `recurring` opt-in, and only when the user has at least one tracked subscription; otherwise there are no extra queries.
- It never writes and never fails the create: a lookup error is logged (ids only) and answers null.
- The `recurring` opt-in now writes `auto_log: true` explicitly, rather than relying on the database default, so Postgres, the test fake and devMock agree.

Changed: **`DELETE /api/categories/:id`** (an existing bug this feature makes worse)

`recurrences.category_id` is `on delete cascade`, and the route reassigns transactions only. So deleting a category silently deletes every subscription and "Repeat this" schedule in it. - Count the category's recurrences beside its transactions. 409 when either is non-zero and no `reassign_to` is given: `{ transactionCount, subscriptionCount }`. - With `reassign_to`, move both transactions and recurrences, scoped by user. - Client (`CategoryManager.jsx`): open the reassign dialog when either count is non-zero, with copy covering both.

Nightly job — tracked rows must never fire

Three independent guards, each tested on its own:

1. The select (`lib/runRecurrences.js`) gains `.eq('auto_log', true)`, and `RECURRENCE_COLUMNS` gains `auto_log`, so the guards below actually see the column.
2. The claim `UPDATE` gains `.eq('auto_log', true)`. `processOne` is exported as a test seam, and it returns **`{ outcome: 'skipped' }`** at the very top unless `row.auto_log === true && row.next_run_at`, before any date maths. Today `advanceToFuture` runs before the claim and would throw on a null date or an `annual` interval. Guard 2 is tested with a stale read: the in-memory row passed to `processOne` says `auto_log: true` with a due `next_run_at`, while the stored row has since become tracked (`auto_log: false`). The claim's `.eq('auto_log', true)` must then match nothing: no claim, no insert. Removing that filter makes the test fail. (A row that is already tracked in memory is stopped by the early return, which tests the early return, not guard 2.)
   - Also, each `processOne` call in `runRecurrences` is wrapped in try/catch, so one bad row counts as an error instead of aborting the sweep for every remaining user that night.
3. `dueRecurrences` (pure) fails closed: it fires only when `r.auto_log === true`, `next_run_at` is set, `cancelled_at` is not set, and `next_run_at <= today`. In SQL, a null `next_run_at` never satisfies `.lte(...)` either.

The `devMock` cron mirrors all three.

Client

- **`components/SubscriptionMarker.jsx`** (new): calls preview with the current values and interval, and renders the existing / detected / pills + likely / maybe states. Used by both entry points. Refetches when the interval or the dialog's values change.
- **`pages/Transactions.jsx`** EditDialog: the Subscription section, with its action carried on `onSave`. The parent's save mutation PATCHes exactly as today, then calls track, attach or

detach. A 409 from **`/track`** keeps the dialog open and shows that 409's `existing` offer in the Subscription section ("Add to Netflix?"), instead of closing with a generic error. This is only reachable if something changed between preview and save. If the second call fails after the PATCH succeeded, the toast says so plainly: "Saved — but couldn't mark it as a subscription. Try again."

- **`pages/Transactions.jsx`** row and filter: a "Subscription" pill when `recurrence_id && !is_recurring`. The Recurring filter matches any `recurrence_id || is_recurring`.
- **`pages/Subscriptions.jsx`**: the "Add a subscription" button and dialog (`track/recent` + search → `SubscriptionMarker` → Track it). The empty-state hint points at both entry points.
- **`components/SubscriptionRow.jsx`**: "Marked by you" / "Logs itself" pills; Remove (confirm) for tracked rows; no amount edit on tracked rows; the existing manual gating stays.
- **`components/QuickAddDialog.jsx`**: after success, the match toast fires after the whole existing chain and the shield-used toast. Yes → `POST …/attach` (with `reactivate` for the "is it back" kind) → `invalidateMoney`.
- Toasts over dialogs. Making the attach toast tappable while a dialog is open takes two changes:
  - `components/ui/toaster.jsx` passes `style={{ pointerEvents: 'auto' }}` to `<Sonner>`. Not in `main.jsx`: its `<Toaster />` takes no props, so a prop there is silently dropped.
  - `components/ui/dialog.jsx` `DialogContent` gets `onInteractOutside={(e) => { if (e.target.closest?.('[data-sonner-toaster]')) e.preventDefault(); }}`. Without it, Radix treats a tap on a toast as "outside" and closes the open dialog, losing a half-typed entry.
  - Manual check: reopen Quick Add within 12 seconds, tap Yes, and confirm the dialog stays open and the charge attaches.
- **`lib/subscriptions.js`**: `CADENCE_LABELS.annual = 'Yearly'`.
- **`CategoryManager.jsx`**: the reassign trigger and copy, as above.
- Every mutation ends in `invalidateMoney(queryClient)`, which already refreshes `subscriptions`, `transactions` and `categories`.

devMock (**`server/scripts/devMock.js`**)

Mirror every change in the same session (BUGS-FIXED shape #2), by name: the six endpoints (preview, recent, track, attach, detach, DELETE), the GET changes, the PATCH change, `subscriptionMatch` and the explicit `auto_log: true` on POST /api/transactions, the category DELETE change, and the cron's three guards. Mirror them with the same 400/404/409 guards as the real routes; existing mock handlers here are looser, and new ones must not copy that. `lib/subscriptionMatch.js`, `lib/recurrences.js` and the shared `addDays`/next-expected helper are imported, not re-implemented, and devMock's own copies of the cadence and cost formulas are deleted. Seed `auto_log` explicitly on every mock recurrence, plus one tracked subscription with one attached charge, one unlinked look-alike near its next date, and one older look-alike, so every new screen has something to show under `npm run dev:mock`. While in this area, also close the existing mock gap where PATCH /api/transactions skips the category/type guard (BUGS-FIXED 2026-09-23 #2).

Tests (server — the client has no test runner)

Harness first (`test/helpers/routeHarness.js`). The fake must not be more forgiving than PostgREST: - insert/update/delete return `data: null` unless `.select()` was chained; - insert ids come from `randomUUID()`; - `select(cols, { count, head })` returns `count`, and `data: null` when `head` is set (the category DELETE route reads `count`; today the fake drops the option, so `count` is undefined); - add a per-table error-injection hook; - an optional `maxRows` cap that mimics PostgREST (silently truncating), so the paging tests can fail if paging is missing; - chained `.order()` calls (today the fake keeps only the last), and a recorded list of `.order()` calls, so a test can assert `id` is the tie-breaker (a stable JS sort would otherwise hide its absence). `maxRows` is applied after range/offset, as PostgREST does.

A review checked the impact: every route write whose result is read already chains `.select`, and no test reads write results directly or references `gen_` ids, so roughly no existing tests should break. Run the suite to confirm before writing any feature code.

Run the full suite after the harness change, before any feature code, and fix any existing test that relied on the old behaviour.

Fixtures: add `auto_log: true` to every existing recurrence fixture (cron, subscriptions and recurrences suites). This keeps them passing against the new guards honestly, without loosening a guard.

- `subscriptionMatch.test.js` (new, pure):
  - `nameStrength`: exact, first word, none; synthetic `auto:` keys count as no key; keys gathered from attached charges;
  - silent vs asking on the same inputs: “Apple Music” vs “Apple TV” is asking-yes, silent-no; a no-note expense at +18% is asking-yes, silent-no; no no-note input is ever a silent match; a named expense never matches an unnamed subscription;
  - the ±30% and ±20% boundaries on both sides; income and linked rows never match;
  - `cycleAligned` per cadence, including a missed cycle, and weekly offsets 3, 4, 10, 11 not aligned;
  - `bestTrackedMatch`: ranking, weekly name-only, zero-charge never; cancelled only on an exact note, dated after the cancellation day (first-word and no-note cancelled → null);
  - `bestPreviewMatch`: zero-charge and cancelled included; `attach` vs `reactivate` by date; `strength`;
  - the cancellation day computed in the user’s zone (a UTC instant just after midnight in London that is still the previous day in New York).
- `recurrences.test.js`: `subscriptionCost` per cadence; `isTracked` fails closed (undefined is not tracked); `dueRecurrences` fires only `auto_log === true` (false, undefined and a null date never fire).
- The cron suite:
  - guard 2 via exported `processOne` with the stale-read setup above: no claim and no transaction; the early return: tracked rows with a null `next_run_at`, and a tracked `annual` row, are skipped without throwing;
  - end to end, a tracked row with a due date is never fired;
  - one row that throws doesn’t stop the other users’ schedules;
  - existing behaviour is unchanged.
- The subscriptions route suite, at all three encryption phases:
  - preview `existing` (attach, reactivate and final-charge offers; `strength`; `needsCate-`

gory) / detected (own note only; “Apple TV” detected does not block marking “Apple Music”) / likely / maybe, using the dialog’s override values;
  - track/recent: with more than 200 newer rows, a 300-day-old search match is still returned;
  - a tracked row with zero attached charges: GET returns 200 with null lastCharged and nextExpected and the stored amount, at all three phases;
  - /track: happy path with stored columns per phase; 409 (with strength: 'exact') when an exact-note tracked match exists even at +300%, then attach and confirm GET’s amount follows; 409 plus no orphan row when the marked transaction is already linked; another user’s ids, income ids and linked ids in includeTransactionIds all left alone; 400 income or no category; 404 for another user’s transaction;
  - attach/detach ownership and conditional 409/404, and attach reactivate;
  - DELETE of tracked vs “Repeat this”;
  - GET: tracking, derived amount, lastCharged and nextExpected (edit a linked charge → the figures follow), yearly cost;
  - suppression: tracked “Netflix” plus detected “netflix” → one row; tracked “Apple Music” with an attached charge noted “Apple” plus detected “apple” (iCloud) → both (absorbed keys are never silent); tracked “Apple Music” plus detected “Apple TV” → both; a tracked no-note subscription plus a detected inferred row → both (the accepted, visible case); a detected named row is never hidden by a no-note tracked one; a cancelled tracked “Netflix” with 3 pre-cancel unlinked “netflix” charges → no active detected row, and one post-cancel charge brings it back;
  - /track 409 on an exact-note match (active or cancelled), and 201 on a first-word or no-note resemblance; two different no-note services both trackable;
  - paging past an injected maxRows with same-day rows at the page edge (none skipped or repeated), for both GET and track/recent;
  - detected rows carry no *_enc, user_id or raw transaction (the clean() leak check);
  - PATCH amount on a tracked row → 400.
- The transactions suite: subscriptionMatch attach, reactivate, outside-window null and no-tracked null; an injected lookup error still returns 201; the opt-in writes auto_log: true.
- A new categories route suite (test/helpers/categoriesRouteSuite.js plus off/dual/enc files, mocking lib/supabase.js like the others; none exists today): a 409 with subscriptionCount only; reassign actually moves recurrences (a positive assertion, so the other-user check cannot pass vacuously); another user’s recurrences untouched.
- Migration test (opt-in real-PostgreSQL style, like the existing .pg tests): apply 014, insert one pre-existing row, then 017a twice. Assert:
  - the second run succeeds, and the existing row reads auto_log = true;
  - auto_log=false with a null next_run_at is accepted (the insert every /track makes);
  - auto_log=true with a null next_run_at is rejected;
  - annual is accepted with auto_log=false, and 'yearly' is rejected.
- npm test stays green, with the real-PostgreSQL tests actually running. Report their count: a green total is not proof the whole suite ran (BUGS-FIXED 2026-09-23).

Docs (same session, per the definition of done)

FEATURES.md (the Subscriptions section; also fixing its stale lines about the recurring executor and Task 6.2.1), ARCHITECTURE.md (schema: `auto_log`, `annual`, nullable `next_run_at`; the six endpoints), BUILD_PLAN.md (Phase 17), SECURITY.md (six new user-scoped endpoints, no new data class), and a BUGS-FIXED.md row for the category-delete cascade on recurrences.

Rollout (order matters)

1. Alex runs `017a_subscription_tracking.sql` in the Supabase SQL editor (exact steps in the hand-off). Then a read-only check confirms that `recurrences.auto_log` exists. The API must not deploy first, or /subscriptions 500s on the new column.
2. Deploy `trim-api`, then the client (the `/deploy` skill's API-first rule).
3. Check: `/api/health` 200; /subscriptions loads; an existing "Repeat this" row shows "Logs itself"; the next morning's job count matches the auto-log schedules that were due.

Out of scope

Renaming tracked subscriptions (the name comes from the note). Changes to Quick Add's "Repeat this", including annual auto-log. Remembering a "No" answer. Attaching without asking. Bank import. Ask Trim awareness. Storing a billing day.

Invariants this must not break

- Tracked subscriptions never create transactions (three guards).
- No new path awards XP. Cron-created transactions still award none.
- "Repeat this" behaviour is unchanged.
- A subscription is never silently hidden. It can be listed twice only in the accepted cases under Known limits, which are always visible and fixable (the double-counting rule above).
- Guards read decoded rows; responses never return raw decoded rows (BUGS-FIXED 2026-09-23 #5).
- A calendar day belongs to the user: matching compares transaction dates (already the user's local days), and `track/recent` takes its window from `lib/userZone.js`.
- The merchant normaliser stays single-sourced in `lib/merchant.js`.

Known limits (accepted)

- A note is only as good as its spelling: "netflx" and "Netflix" do not match by name. With no note, matching falls back to category and amount, which is why every attach asks first.
- A subscription can still be listed twice when Trim cannot be sure, and it deliberately does not guess, because a wrong guess would hide a different subscription invisibly:
  - a no-note tracked subscription whose later charges Alex declined or missed three times can also appear as an "inferred" detected row;
  - a different note on a price-jumped charge (e.g. the note changed and the price rose more than 30%).

  Both are visible. The right fix is to attach the charges (the amount then follows the newest one). "Not a subscription" on the detected row hides the duplicate but would leave the tracked amount stale, so the copy points at attaching first.

- Two concurrent subscriptions with the identical note (e.g. two separate “Apple” plans both noted “Apple”) are treated as one. To keep them apart, give one a different note (“Apple TV”).
- `normaliseMerchant` keeps two words and strips diacritics (“Żabka” → “abka”). This is existing behaviour, inherited.
- With a missed cycle, the cycle window allows about 0.44 days of drift per month (calendar months vs 30 days), so after roughly 11 missed months a charge may no longer line up and simply won’t prompt. It can still be attached from the Edit dialog.